MLIR as inline assembly

Using mlir! blocks in place of inline asm and compiler builtins, to cut the assembly code in libraries

count_ones — math dialect

```
fn count_ones(x : i32) → i32 {
  return mlir!(
    inputs : (%a = x : i32),
    returns : i32,
    dialects : ["math"]
  ) {
    %r = math.ctpop %a : i32
    macro.yield %r : i32
  };
}
```

ABOUT VX

A language for tensor programs

Example from the blog post

```
let x = transfer(x_host, Memory::GPU_HBM);
let mut y = transfer(y_host, Memory::GPU_HBM);

spawn on(Topology::GPU) {
  for i in 0..8 {
    y[i] = 2.0 * x[i] + y[i];
  }
}

let result = transfer(y, Memory::CPU_DRAM);
print(result);
```

SYNTAX

Close to Rust: fn, let, match, impl and macro_rules.

TENSORS

Element type and shape are part of the type. a @ b becomes one linalg.matmul.

PLACEMENT

Where a value lives is part of its type too. Printing y on the CPU without the copy back is a type error.

OPEN SOURCE

Apache 2.0 with LLVM exceptions, at github.com/vx-lang/Vx.

ABOUT VX

How Vx uses MLIR

`Vx source`	Element type, shape and placement are part of a tensor’s type.
↓	
`Flat HIR`	Vx’s own IR, after type and borrow checking. A library can ship as a .vxlib file.
↓	
`vx dialect`	Defined in TableGen. vx.spawn and vx.transfer record where code runs and where data lives.
↓	
`Upstream dialects`	linalg, scf, vector, math, gpu and llvm. **mlir! blocks are written in these.**
↓	
`LLVM IR and PTX`	LLVM IR for the CPU. PTX for the GPU, through MLIR’s NVVM path.

THE PROBLEM

Three ways a library can write backend specific code

WAY 1

Inline assembly

```
asm("popcnt %1, %0"
    : "=r"(r)
    : "r"(x));
```

The optimizer cannot look inside it. Each target needs its own string.

WAY 2

Compiler builtins

```
__builtin_popcount(x)
__atomic_fetch_add(p, 1,
  __ATOMIC_SEQ_CST)
```

The optimizer understands them. Each new one is a change to the compiler.

WAY 3

Platform libraries

```
#include <math.h>
double y = sqrt(x);
float z = expf(w);
```

A call into code written in C or assembly.

4TH APPROACH the mlir! block.

THE PROBLEM

Analysis of each approach

	Inline asm	Builtin	Library call	mlir! block
The optimizer sees the operation	**No**	**Yes**	**Some, by name**	**Yes, after inlining**
Works on several targets	**No**	**Yes**	**Where ported**	**Where the dialect lowers**
Who adds a new one	Anyone	Compiler engineer	Anyone	**Anyone**
Needs libc at run time	**No**	**No**	**Yes**	**No, if each op has an instruction**

An mlir! block is inline assembly whose instructions are MLIR operations.

vxc parses it, verifies it, inlines it and optimizes it **with** the code around it.
Acts like a more powerful bulitin

THE DESIGN

Example: count the set bits

count_ones — math dialect

```
fn count_ones(x : i32) → i32 {
  return mlir!(
    inputs : (%a = x : i32),
    returns : i32,
    dialects : ["math"]
  ) {
    %r = math.ctpop %a : i32
    macro.yield %r : i32
  };
}
```

`inputs`

Binds a Vx expression to a named SSA value with an MLIR type.

`returns`

The Vx type of the block's value. A block that only writes leaves it out.

`clobbers`

The Vx variables the block writes, such as **self** in a tensor's fill.

`dialects`

The dialects the body uses.

`macro.yield`

Hands the result back to Vx.

THE DESIGN

Side by side with GCC inline assembly

	GCC inline asm, C	Vx mlir! block
Instructions	A template string for one ISA	**MLIR operations from any dialect**
Inputs	`: "r"(x)`	`inputs : (%a = x : i32)`
Outputs	`: "=r"(r)`	`returns : i32` `macro.yield %r : i32`
Clobbers	`: "cc", "memory"`	`clobbers : [self]`
Operands	%0, %1 by position, with constraint letters	**%a, %r by name, with MLIR types**
Checked by	The assembler, after code generation	**MLIR's parser and verifier**

THE DESIGN

How vxc compiles a block

01 · EXPAND

The macro expander reads the fields. The body stays as text.

02 · CHECK

The type checker checks each input. The block's value gets the type in returns.

03 · WRAP

The code generator writes a private func.func and a call. macro.yield becomes return.

04 · INLINE

MLIR parses and verifies the module. At -O1 and above, vx-inline moves the body into its caller.

THE DESIGN

What count_ones becomes

emit-mlir -O0 — a private function and a call

```
func.func @count_ones(%arg0: i32) → i32 {
  %0 = call @vx_flat_mlir_0(%arg0) : (i32) → i32
  return %0 : i32
}
func.func private @vx_flat_mlir_0(%arg0: i32) → i32 {
  %0 = math.ctpop %arg0 : i32
  return %0 : i32
}
```

Output of vxc --action emit-mlir. MLIR prints the block's %a as %arg0.

emit-mlir -O2 — inlined into the caller

```
func.func @count_ones(%arg0: i32) → i32 {
  %0 = math.ctpop %arg0 : i32
  return %0 : i32
}
```

The wrapper is an ordinary function, so MLIR's inliner needs no special case.

THE DESIGN

The optimizer sees inside the block

3 → 1

loops for these three calls on 64×64 tensors, at -O1 and above

```
a.fill(x); b.fill(x); a.compare(&b)
```

WHY THE LOOPS FUSE

fill and compare are mlir! blocks written with linalg. After vx-inline, linalg becomes affine loops, and the affine passes fuse them.

WHAT INLINE ASM WOULD DO

Each fill would stay a separate loop that no pass can look into.

IN THE LIBRARY

Builtins moved from the compiler to the library

Module	Blocks	MLIR operations	In C, this is
`core::num`	56	`math.ctpop, ctlz, cttz, exp`	__builtin_popcount
`core::sync::atomic`	53	`llvm.atomicrmw, cmpxchg`	__atomic_* builtins
`std::tensor`	28	`linalg, scf, vector`	SIMD intrinsics
`core::libm`	19	`math.fma, math.sqrt, bitcast`	the platform libm
`core::ptr`	3	`llvm.getelementptr`	pointer arithmetic
`std::alloc`	3	`vx.heap_alloc, vx.heap_grow`	malloc and realloc
`std::rand`	1	`math.sqrt`	a call to libm
`core::hint`	1	`llvm.intr.assume`	__builtin_assume

164 blocks, counted in the source outside comments. Macros stamp many of them once per integer or float width.

IN THE LIBRARY

Atomics without compiler intrinsics

stdlib/core/sync/atomic.vx — llvm dialect

```
match order { // struct AtomicI32 { v : SharedCell<i32> }
  AtomicOrdering::Relaxed ⇒ {
    return mlir!(
      inputs : (%p = &mut self.v.get() : !llvm.ptr, %x = val : $mlir),
      returns : $t,
      dialects : ["llvm"]
    ) {
      %r = llvm.atomicrmw $op %p, %x monotonic : !llvm.ptr, $mlir
      macro.yield %r : $mlir
    };
  }
  // Acquire, Release, AcqRel, SeqCst: one block each
}
```

01

Each method is one LLVM atomic op. The module needs no libc and no compiler intrinsic.

02

MLIR writes the ordering into the op's text, so each ordering gets its own block, chosen by a match.

03

With a constant ordering, the optimizer keeps one case. A test checks for one atomic op per call.

IN THE LIBRARY

Generic blocks through macro substitution

stdlib/std/tensor.vx — vector dialect

```
macro_rules stamp_tensor_sums {
  ($t : ident , $lanes : expr) => {
    impl Tensor<$t, [?]> {
      fn sum(self : &Tensor<$t, [?]>) -> $t {
        return mlir!(inputs : (%a = self : memref<?x$t>), returns : $t, dialects : [...]) {
          // four vector accumulators, then a scalar loop over the elements left over
          %x0 = vector.load %a[%i0] : memref<?x$t>, vector<$lanes x $t>
          %s0 = arith.addf %v0, %x0 : vector<$lanes x $t>
          %head = vector.reduction <add>, %p fastmath<reassoc> : vector<$lanes x $t> into $t
          ...
stamp_tensor_sums!(f32 , 16);
stamp_tensor_sums!(f64 , 8);
```

$t and $lanes are replaced in the text before MLIR parses it.

f32 gets 16 lanes and f64 gets 8. LLVM fits them to each target.

A C library writes this loop with SIMD intrinsics, once per ISA.

IN THE LIBRARY

One block, lowered for each target

stdlib/core/num.vx — math dialect

```
fn fast_exp(self : $t) -> $t {
  return mlir!(
    inputs : (%a = self : $t),
    returns : $t,
    dialects : ["math"]
  ) {
    %r = math.exp %a : $t
    macro.yield %r : $t
  };
}
```

ON A GPU

math.exp becomes a few fast instructions in the type's own width.

ON THE CPU

LLVM lowers the same op, which can mean a call to libm.

INSIDE A KERNEL

core declares no extern, so a `spawn on(GPU)` region can call fast_exp. The kernel takes the functions it calls along.

CHOOSING A BLOCK

if comptime Topology::Current == ... picks a block while the program compiles, as #if does around inline asm.

LIMITS

Errors the programmer sees

count_ones — type error

```
fn count_ones(x : i32) → i32 {
  return mlir!(inputs : (%a = x : i32), returns : i32, dialects : ["math"]) {
    %r = math.ctpop %a : f32    // %a is an i32
    macro.yield %r : i32
  };
}
```

vxc output — built on 24 October 2026

```
The `mlir!` block at line 2, column 16 is not valid MLIR:
  use of value '%a' expects different type than prior uses: 'f32' vs 'i32'
```

MLIR's parser finds the error. vxc reports it once, with the line and column where the block starts.

The message does not point at the line inside the block yet. In a long block, that matters.

LIMITS

What does not work yet

01

dialects is not checked

vxc parses the list but trusts MLIR to do the error checking #2332

02

Errors arrive late

MLIR parses a block when vxc generates code for it.

03

Errors point at the block

The message gives the block’s line and column, not the line inside it.

04

No choice by ISA

spin_loop needs pause on x86 and yield on Arm. core cannot pick an instruction by target yet.

05

Pointers are trusted

When a block takes a raw pointer, the compiler cannot tell which tensor it points into. It trusts the author.

06

Some ops need libm

math.asinh, acosh, atanh and erf become libm calls. core::libm ports LLVM libc to Vx instead.

FOR DISCUSSION

Open questions for MLIR

Q1

A shared form for inline MLIR

Named operands, a yield and declared effects. Could frontends that embed MLIR text agree on one form?

Q2

Effects on the wrapper

Can a block’s clobbers become memory effects that passes see before inlining?

Q3

Locations inside the text

How should an error in a block’s text point at the line in the host file?

Q4

Instructions chosen by target

How should one block say “pause on x86, yield on Arm”?

Questions?

vxlang.org/blog/vx-fully-embraces-ml-ir-mlir.html

github.com/vx-lang/Vx

FILES TO READ

stdlib/core/num.vx	integer methods
stdlib/core/sync/atomic.vx	atomics
stdlib/std/tensor.vx	tensor reductions
stdlib/core/libm.vx	the libm port